

Relatório Técnico — Laboratório de Circuitos

Projeto Integrador — Parte II

Unidade de Controle (cabeada e microprogramada)

Campo	Preenchimento
Disciplina	Arquitetura e Organização de Computadores — Semestre 2026.2
Professor	Prof. Dr. Herbert Oliveira Rocha
Integrante 1	Mateus Henrique de Souza Pinheiro
Integrante 2	Darckison Almeida Trajano
integrante 3	Fabício Cauã Pereira Silvino
Repositório GitHub	https://github.com/MateusHenrique2025/AOC_MateusHenriqueDarckisonAlmeidaFabricao_UFRR_LabCircuitos_2026
Ferramenta	Logisim-Evolution v5.0.0
Data de referência	01/10/2026

0. Situação atual e limitações

Este relatório é parcial. A Parte II está incompleta: a unidade de controle cabeada está pronta e testada, mas o caminho de dados e a versão microprogramada ainda não foram montados. Os itens da Seção 0.3 **não estão entregues** nesta versão.

0.1 O que está concluído

Item	Situação
Máquina de estados de 5 estados com 3 flip-flops D (só AND, OR, NOT)	Concluído e testado
9 sinais de controle (11 bits) em portas, sem ROM nem tabela de verdade da biblioteca	Concluído e testado
Subcircuito UC_cabeada (entradas RESET, CLK, B; saídas SF2...SF0 e os sinais de controle)	Concluído
Estado atual visível em display (JunMain)	Funciona, com a ressalva da Seção 9.2
Tabela de transição, tabela de tempo, mapas de Karnaugh, equações, contagem de portas e CPI	Feitos neste documento
Microprograma (Tabela 10)	Calculado neste documento; ainda não carregado em circuito

0.2 Onde a equipe parou

O trabalho parou no **Passo 4B da montagem do caminho de dados** (circuito CPU). Estão montados:

- a instância de `UC_cabeada` e os 11 túneis de sinais de controle;
- o **PC** (registrador de 8 bits), ainda com a entrada `D` presa a uma constante provisória `0x04`, no lugar do `PC + 4` vindo da ULA;
- a **ROM 16×8** de programa, endereçada pelos 4 bits baixos do PC por um distribuidor (túnel `END_ROM`), com `0x11` no endereço 0 e `0x22` no endereço 4 (conteúdo de teste);
- o **IR** (registrador de 8 bits com `WE` em `IRWrite`);
- os túneis de 2 bits `ALUSrcB` e `ALUOp`, formados por distribuidores.

Em andamento: o multiplexador 2→1 da entrada A da ULA já foi colocado, mas faltam as ligações (entrada 0 = `PC`, entrada 1 = constante `0x00` provisória, seleção = `ALUSrcA`, saída = túnel `ULA_A`). Falta também renomear para `IR` o túnel da saída do registrador do IR, que ainda está com o rótulo `PC` e gera conflito com o túnel do PC.

0.3 O que ainda falta (limitação deste relatório)

Componente / entregável	Situação
Multiplexador da entrada A (Comp. 02)	Parcial (Seção 0.2)
Multiplexador 4→1 da entrada B (Comp. 02)	Não iniciado
ULA de 8 bits (Comp. 10) e saída <code>Zero</code>	Não iniciado
Registrador <code>ULAOut</code> e multiplexador do <code>PCSource</code>	Não iniciado
Somador com constante 4 (Comp. 04)	Não iniciado; hoje o <code>PC + 4</code> é uma constante provisória
Banco de registradores (Comp. 07) e registradores A e B	Não iniciado
Extensor de sinal 4→8 bits (Comp. 11) e deslocamento de 2	Não iniciado
Lógica de escrita do PC (<code>PCWrite</code> OU <code>PCWriteCond E Zero</code>)	Não iniciado
Formato da instrução de 8 bits (opcode, rs, rt, rd, imediato)	Não definido; B é entrada manual
<code>parte2_microprogramada.circ</code> (<code>μPC</code> , ROM de microcódigo, despacho, decodificação)	Não iniciado
<code>microcodigo.txt</code>	Conteúdo calculado (Seção 6); arquivo ainda não gerado
<code>tabela_tempo.xlsx</code> OU <code>.pdf</code>	Conteúdo pronto (Seção 4); falta exportar
Testes T-07, T-08 e T-09	Não realizados
Teste T-06 completo (IR = <code>0x11</code> no CPU)	Parcial (Seção 8)
Seção 7 com números medidos da versão microprogramada	
Componente 09 (detector da sequência "101")	

Como a versão microprogramada reaproveita o caminho de dados, os testes T-07 a T-09 só podem ser feitos depois que a ULA, o banco de registradores e os multiplexadores estiverem ligados.

1. Introdução e objetivos

Esta parte do projeto constrói a unidade de controle de um caminho de dados multiciclo de 8 bits, no estilo MIPS reduzido, com cinco instruções (`add`, `sub`, `and`, `or`, `beq`). Os objetivos de aprendizagem tratados são:

- **OA5:** implementar uma máquina de estados de cinco estados que gere os sinais de controle do caminho de dados.
- **OA6:** minimizar por mapas de Karnaugh as funções de próximo estado e derivar as equações booleanas.
- **OA7:** escrever o microprograma de oito bits equivalente à unidade cabeada e compará-lo com ela.

2. Declaração de uso de ferramentas de IA generativa

Foi utilizada a ferramenta **Claude (Anthropic)** nas seguintes etapas e finalidades: (a) explicação do enunciado da Parte II.

3. Unidade de controle cabeada

3.1 Sinais de controle

Sinal	Bits	Função
PCWrite	1	Escreve no PC incondicionalmente
PCWriteCond	1	Escreve no PC apenas se a ULA indicar Zero
PCSource	1	0 = saída da ULA; 1 = ULAOut
MemRead	1	Habilita a leitura da ROM de programa
IRWrite	1	Carrega o registrador de instrução
RegWrite	1	Habilita a escrita no banco de registradores
ALUSrcA	1	0 = PC; 1 = registrador A
ALUSrcB	2	00 = registrador B; 01 = constante 4; 10 = extensão de sinal deslocada de 2
ALUOp	2	00 = soma; 01 = subtração; 10 = definida pelo campo funct

Tabela 1 — Sinais de controle (nove sinais, onze bits).

3.2 Estados e diagrama

Estado	Código (Q2Q1Q0)	Nome	O que acontece
S0	000	Busca	$IR \leftarrow \text{Mem}[PC]$; $PC \leftarrow PC + 4$
S1	001	Decodificação	$A \leftarrow \text{Reg}[\text{rs}]$; $B \leftarrow \text{Reg}[\text{rt}]$; $\text{ULAOut} \leftarrow PC + (\text{ext}(\text{imm}) \ll 2)$
S2	010	Execução tipo-R	$\text{ULAOut} \leftarrow A \text{ op } B$
S3	011	Conclusão tipo-R	$\text{Reg}[\text{rd}] \leftarrow \text{ULAOut}$
S4	100	Conclusão de desvio	$A - B$; se Zero, $PC \leftarrow \text{ULAOut}$

Tabela 2 — Os cinco estados.

A entrada **B** indica o resultado da decodificação do opcode: **B = 1** para beq e **B = 0** para tipo-R. Só importa em S1.

```

S0 → S1          (sempre)
S1 → S2    se B = 0    (tipo-R)
S1 → S4    se B = 1    (beq)
S2 → S3
S3 → S0
S4 → S0

```

Figura 1 — Transições entre estados.

3.3 Tabela de transição

Estado atual (Q2Q1Q0)	B	Estado seguinte (D2D1D0)
S0 = 000	X	001
S1 = 001	0	010
S1 = 001	1	100
S2 = 010	X	011
S3 = 011	X	000
S4 = 100	X	000

Tabela 3 — Tabela de transição (cinco linhas úteis).

Estados não usados (101, 110, 111). Foram tratados como **condições indiferentes**, o que gera equações menores. Com as equações da Seção 3.6, cada estado não usado volta a um estado válido em um ciclo:

Estado não usado	Próximo estado
101	010 (se B = 0) ou 100 (se B = 1)
110	010
111	000

Tabela 4 — Recuperação dos estados não usados.

A máquina não trava em nenhum deles.

3.4 Tabela verdade dos sinais de controle

Cada sinal depende só do estado (cinco linhas). X indica condição indiferente.

Estado	PCWr	PCWrC	PCSrc	MemRd	IRWr	RegWr	SrcA	SrcB	ALUOp
S0 (000)	1	0	0	1	1	0	0	01	00
S1 (001)	0	0	X	0	0	0	0	10	00
S2 (010)	0	0	X	0	0	0	1	00	10
S3 (011)	0	0	X	0	0	1	X	XX	XX
S4 (100)	0	1	1	0	0	0	1	00	01
101, 110, 111	X	X	X	X	X	X	X	XX	XX

Tabela 5 — Tabela verdade dos sinais (também é a tabela de tempo da Seção 4).

3.5 Mapas de Karnaugh

Mapas das três funções de próximo estado (variáveis Q2Q1 nas linhas e Q0B nas colunas, em código Gray) e de três sinais de controle (Q2 nas linhas e Q1Q0 nas colunas).

D2 = Q1'·Q0·B. O agrupamento é o par de células (linha 00, coluna 11) e (linha 10, coluna 11), aproveitando o X.

Q2Q1 \ Q0B	00	01	11	10
00	0	0	1	0
01	0	0	0	0
11	X	X	X	X
10	0	0	X	X

D1 = Q1·Q0' + Q1'·Q0·B'. Dois grupos: linhas 01 e 11 nas colunas 00 e 01 (quatro células); linhas 00 e 10 na coluna 10 (par com X).

Q2Q1 \ Q0B	00	01	11	10
00	0	0	0	1
01	1	1	0	0
11	X	X	X	X
10	0	0	X	X

D0 = Q2'·Q0'. Grupo de quatro células: linhas 00 e 01, colunas 00 e 01.

Q2Q1 \ Q0B	00	01	11	10
00	1	1	0	0
01	1	1	0	0
11	X	X	X	X
10	0	0	X	X

RegWrite = Q1·Q0. Par (011, 111), usando o X de 111.

Q2 \ Q1Q0	00	01	11	10
0	0	0	1	0
1	0	X	X	X

ALUSrcA = Q2 + Q1. Dois grupos: linha Q2 = 1 inteira, e colunas 11 e 10.

Q2 \ Q1Q0	00	01	11	10
0	0	0	X	1
1	1	X	X	X

ALUSrcB1 = Q0. Grupo: colunas 01 e 11, nas duas linhas.

Q2 \ Q1Q0	00	01	11	10
0	0	1	X	0

Q2 \ Q1Q0	00	01	11	10
1	0	X	X	X

Figuras 2 a 7 — Mapas de Karnaugh de D2, D1, D0, RegWrite, ALUSrcA e ALUSrcB1

3.6 Equações finais (soma de produtos)

Próximo estado

- $D2 = Q1' \cdot Q0 \cdot B$
- $D1 = Q1 \cdot Q0' + Q1' \cdot Q0 \cdot B'$
- $D0 = Q2' \cdot Q0'$

Sinais de controle

Sinal	Equação
PCWrite = MemRead = IRWrite = ALUSrcB0	$Q2' \cdot Q1' \cdot Q0'$
PCWriteCond = PCSource = ALUOp0	$Q2$
RegWrite	$Q1 \cdot Q0$
ALUSrcA	$Q2 + Q1$
ALUSrcB1	$Q0$
ALUOp1	$Q1$

Tabela 6 — Equações dos sinais de controle.

Valores adotados nas condições indiferentes. O circuito assume: PCSource = 0 em S1, S2 e S3 (o PC não é escrito nesses estados); em S3, ALUSrcA = 1, ALUSrcB = 10 e ALUOp = 10 (a ULA não é usada nesse ciclo, pois Reg[rd] recebe ULAOut).

Justificativa dos X. PCSource só importa quando o PC pode ser escrito (S0 e S4). Em S3 apenas se escreve o registrador de destino com ULAOut, então ALUSrcA, ALUSrcB e ALUOp não têm efeito. Os sinais que escrevem algo (PCWrite, PCWriteCond, RegWrite, IRWrite) nunca são X, porque precisam valer 0 para não escrever por engano.

3.7 Implementação no Logisim-Evolution

- **Subcircuito `UC\_cabeada`.** Entradas: RESET, CLK, B. Saídas: SF2, SF1, SF0 (estado) e os sinais da Tabela 1 (ALUSrcB e ALUOp com os dois bits em pinos separados, reunidos por distribuidores no circuito CPU).
- **Registrador de estado:** três flip-flops D (F2, F1, F0) com clock comum e RESET no clear. As saídas negadas dos flip-flops foram aproveitadas como nQ2, nQ1, nQ0.
- **Lógica:** somente AND, OR e NOT, ligadas por túneis. Não foram usadas ROM nem tabela de verdade da biblioteca.
- **Clock:** nos circuitos JunMain e CPU, o clock foi implementado com um pino manual, para controlar cada borda de subida nos testes. O componente Clock não foi usado porque pulsava com os ticks automáticos.
- **Estado em display:** SF2 . . SF0 formam um barramento de 4 bits (bit 3 em 0) que alimenta o display (Seção 9.2).

Pinos do bloco `UC\_cabeada`.

Pino	Tipo	Descrição
RESET	Entrada, 1 bit	Leva a máquina ao estado 000
CLK	Entrada, 1 bit	Relógio (borda de subida)
B	Entrada, 1 bit	1 = beq, 0 = tipo-R
SF2, SF1, SF0	Saída, 1 bit cada	Estado atual
PCWrite ... ALUOp1	Saída, 1 bit cada	Sinais da Tabela 1

Tabela 7 — Pinos de UC_cabeada.

3.8 Contagem de portas

Tipo	Quantidade	Onde
AND	6	D2 (1), D1 (2), D0 (1), Q2'Q1'Q0' (1), RegWrite (1)
OR	2	D1 (1), ALUSrcA (1)
NOT	1	nB
Total de portas	9	
Flip-flops D	3	Registrador de estado

Tabela 8 — Contagem de portas.

O número é baixo porque se aproveitaram as saídas negadas dos flip-flops e o termo Q2'Q1'Q0' foi compartilhado entre quatro sinais.

4. Tabela de tempo, ciclos e CPI

A tabela de tempo coincide com a Tabela 5.

Ciclos por classe de instrução

- Tipo-R: S0, S1, S2, S3 = **4 ciclos**
- beq: S0, S1, S4 = **3 ciclos**

CPI médio (70% tipo-R e 30% beq):

$$\text{CPI} = 0,7 \times 4 + 0,3 \times 3 = 2,8 + 0,9 = \mathbf{3,7}$$

5. Caminho de dados (montagem parcial)

Bloco	Pinos principais	Situação
PC (registrador 8 bits)	D (hoje constante 0x04), WE = PCWrite, CLK, R = RESET, Q → túnel PC e distribuidor	Montado; D provisório
Distribuidor do PC	Entrada 8 bits; saída 0 = bits 3–0 (túnel END_ROM); saída 1 = bits 7–4 (sem uso)	Montado
ROM 16×8	Endereço 4 bits (END_ROM); dado 8 bits	Montada; 0x11 em 0 e 0x22 em 4
IR (registrador 8 bits)	D = saída da ROM, WE = IRWrite, CLK, R = RESET	Montado
Mux 2→1, entrada A da ULA	Entrada 0 = PC; entrada 1 = A (provisório 0x00); seleção = ALUSrcA	Em andamento

Bloco	Pinos principais	Situação
Demais blocos	—	Ver Seção 0.3

Tabela 9 — Estado do caminho de dados.

6. Microprograma equivalente (especificado, não implementado)

Formato da microinstrução: ULAOp | Fonte | Ação | Seq, dois bits cada.

Rótulo	End.	ULAOp	Fonte	Ação	Seq	Hex
Busca	0	00	00	01	00	0x04
Decod	1	00	10	00	10	0x22
ExecR	2	10	01	00	00	0x90
ConclR	3	00	01	10	01	0x19
ConclBEQ	4	01	01	11	01	0x5D

Tabela 10 — Microprograma (posições 5, 6 e 7 livres).

Observação sobre ConclR. O campo Fonte não pode ser 00, porque, pela convenção do enunciado, 00 implica escrita incondicional no PC. Foi usado 01.

Conteúdo de `microcodigo.txt`:

```
v3.0 hex words plain
04 22 90 19 5d 00 00 00
```

Lógica de sequenciamento prevista. O μ PC de 3 bits endereça a ROM. O próximo endereço é escolhido por um multiplexador comandado pelo campo Seq: 00 $\rightarrow \mu$ PC + 1; 01 $\rightarrow$ 000; 10 $\rightarrow$ 100 se beq e 010 se tipo-R. Os campos Fonte e Ação são decodificados nos sinais da Tabela 1.

7. Comparação entre as duas versões (rascunho, 300–400 palavras)

A versão cabeada consumiu 9 portas lógicas (6 AND, 2 OR e 1 NOT), além de 3 flip-flops D, porque aproveitou as saídas negadas dos flip-flops e compartilhou o termo $Q_2'Q_1'Q_0'$ entre quatro sinais. A versão microprogramada usa uma ROM de 8 posições por 8 bits, ou seja, 64 bits de armazenamento, dos quais apenas 5 posições (40 bits) são ocupadas. Soma-se a isso um μ PC de 3 bits, um multiplexador de próximo endereço e a lógica que decodifica os campos em sinais. Em portas, a cabeada é menor; a microprogramada troca portas por memória e por sequenciamento.

Sobre o tempo de projeto. Na cabeada o trabalho se concentrou em montar a tabela de transição, desenhar seis mapas de Karnaugh e converter as equações em portas, e um erro no início da cadeia obriga a refazer tudo. Na microprogramada o projeto se resume a preencher a Tabela 10 e carregar cinco bytes, mas exige o hardware de sequenciamento e a decodificação dos campos, que a cabeada não tem.

Se fosse preciso acrescentar uma quinta operação à ULA, o impacto depende do caso. Se ela for selecionada pelo campo funct (ALUOp = 10), nenhuma equação da unidade cabeada e nenhuma microinstrução precisam mudar, pois só o controle local da ULA se altera. Se a operação exigir um estado próprio, a cabeada passa a ter um sexto estado (ainda cabe em 3 bits): as três equações

de próximo estado (D2, D1, D0) precisam ser refeitas, além dos sinais cujo valor no novo estado difere do que as equações atuais já produzem, até nove no pior caso. Na microprogramada bastaria escrever uma ou duas microinstruções novas e ajustar o despacho.

Em um projeto real, a unidade cabeada é preferível quando velocidade e área importam e o conjunto de instruções é pequeno e estável, como em controladores dedicados e processadores RISC simples. A microprogramada é preferível quando o conjunto de instruções é grande e complexo, ou quando é desejável corrigir ou estender o comportamento sem refazer o hardware, como em processadores CISC com atualização de microcódigo.

8. Testes

ID	O que se verifica	Entradas	Saída esperada	Situação
T-06	Ciclo de busca	Reset e um pulso de relógio	Estado vai de S0 para S1; IRWrite pulsado; IR = 0x11	Parcial. Na UC: IRWrite = 1 em S0 e 0 em S1 verificados. No CPU: falta a captura com IR = 0x11 e PC = 0x04
T-07	Execução tipo-R	add na ROM de programa	Quatro ciclos; rd recebe o resultado	Não realizado (depende de ULA e banco de registradores)
T-08	Desvio tomado e não tomado	beq com operandos iguais e depois diferentes	PC muda apenas no primeiro caso	Não realizado
T-09	Equivalência das duas versões	Mesmo programa nas duas	Mesma sequência de sinais, ciclo a ciclo	Não realizado (versão microprogramada não montada)

Tabela 11 — Casos de teste obrigatórios da Parte II.

Testes adicionais já realizados na unidade cabeada.

Teste	Procedimento	Resultado
Sequência com B = 0	RESET, pulsos de clock	000 → 001 → 010 → 011 → 000
Sequência com B = 1	RESET, pulsos de clock	000 → 001 → 100 → 000
Sinais por estado	Percorrer os 5 estados e comparar com a Tabela 5	Valores coincidem (nos X, vale o valor da equação)
Display do estado	Percorrer 0, 1, 2, 3 e 4	Mostra o dígito correspondente (Seção 9.2)

Tabela 12 — Testes da unidade cabeada.

9. Entregáveis e componentes

9.1 Entregáveis da Parte II

Entregável	Critério de aceitação	Situação
parte2_cabeada.circ	Cinco estados com três flip-flops D e lógica minimizada; sem ROM nem tabela de verdade da biblioteca; estado visível	Atendido, com a ressalva da Seção 9.2
parte2_microprogramada.circ	µPC de 3 bits funcional, ROM de microcódigo carregada, despacho operante	Não iniciado
microcodigo.txt	Conteúdo da ROM coerente com a Tabela 10	Conteúdo definido (Seção 6); arquivo a gerar
tabela_tempo.xlsx ou .pdf	Cinco estados, nove colunas, X justificados, CPI	Conteúdo pronto (Seção 4); falta exportar

Entregável	Critério de aceitação	Situação
Seção do relatório	Diagrama de estados, tabela de transição, Karnaugh desenhados, equações, microprograma, comparação	Parcial (este documento)

Tabela 13 — Entregáveis da Parte II (Tabela 19 do enunciado).

9.2 Ressalva sobre o Componente 16 (decodificador de 7 segmentos)

O decodificador de 7 segmentos desenvolvido pela equipe na Parte I apresentava um erro de ligação nos segmentos d e g (o dígito 3 aparecia como 7), que foi **corrigido**. No circuito `JunMain` atual, a exibição do estado usa o **display hexadecimal da biblioteca do Logisim**, que já contém o decodificador.

9.3 Componentes usados na Parte II

Nº	Componente	Situação na Parte II
01	Flip-flop D	Usado (3 flip-flops de estado)
02	Multiplexador de 4 entradas	Parcial (mux da entrada A em andamento)
04	Somador com constante 4	Pendente
05	ROM de 8 bits	Usada como ROM de programa; ROM de microcódigo pendente
07	Banco de registradores	Pendente
09	Detector da sequência "101"	Pendente
10	ULA de 8 bits	Pendente
11	Extensor de sinal 4→8	Pendente
12	Máquina de estados com portas	Concluído
13	Contador síncrono (μPC)	Pendente
15	Otimização por Karnaugh	Concluído (Seção 3.5)
16	Decodificador de 7 segmentos	Parcial (Seção 9.2)

Tabela 14 — Situação dos componentes da Parte II.

10. Conclusão (parcial)

A unidade de controle cabeada foi projetada a partir da tabela de transição, minimizada por mapas de Karnaugh e implementada com 9 portas e 3 flip-flops D, com a sequência de estados e os nove sinais de controle verificados no simulador. O caminho de dados está em montagem (busca de instrução montada), e a versão microprogramada está apenas especificada. As pendências da Seção 0.3 impedem, por ora, os testes T-07 a T-09 e a comparação com números medidos.

Referências

- Enunciado: Laboratório de Circuitos — Projeto Integrador, versão introdutória. Prof. Dr. Herbert Oliveira Rocha, UFRR, 2026.2.
- PATTERSON, D. A.; HENNESSY, J. L. Organização e Projeto de Computadores: a interface hardware/software. (Caminho de dados multiciclo e controle.)
- Documentação do Logisim-Evolution.